Design Verification Report

RISC-V Single-Cycle CPU

Top-Level Integration Testbench

Design Author: Elliot Staresinic

DUT: sc_cpu_top_level | Stimulus: program.mem (30 instructions, 34 retired)
Simulator: Icarus Verilog (iverilog -g2012)
Date: 2026-08-02 | Status: PASS

VERIFICATION RESULT: PASS (0 errors)

SILICON FROM SCRATCH

Contents

1. Executive Summary

This report documents the functional verification of a single-cycle RISC-V CPU at the top level (module sc_cpu_top_level), exercised with a 30-instruction program and checked against two independent reference oracles.

```
Result: PASS, 0 errors. 34 instructions retired before clean halt from a 30-instruction image,
the difference being a three-iteration loop and two instructions skipped by taken branches.
The design terminated without watchdog intervention.
```

The program is shared byte for byte with the pipelined implementation of the same ISA subset, so the two designs are verified on identical work. Coverage gaps are listed in Section 6.

2. Design Under Test

The DUT is a non-pipelined, single-cycle RV32I-subset processor using a Harvard architecture (separate instruction and data memories). Each instruction completes in one clock cycle. The program counter is reset asynchronously and advances on the rising clock edge. The top level also carries four dbg_* observability outputs (dbg_pc, dbg_wb_addr, dbg_wb_data, dbg_wb_enable). These have no functional role and the testbench does not read them; they exist because a module whose only ports are clk and reset has no primary outputs, so synthesis dead-code elimination removes the entire CPU.

2.1 Supported Instruction Subset

Category	Instructions	Notes
R-type	add, sub, and, or, slt	funct7 bit 5 selects sub vs. add
I-type	addi, andi, ori, slti	Immediate arithmetic; slti decodable
Load / Store	lw, sw	Word-aligned, computed base + offset
Branch	beq, bne, blt	One-hot branch control; PC-relative

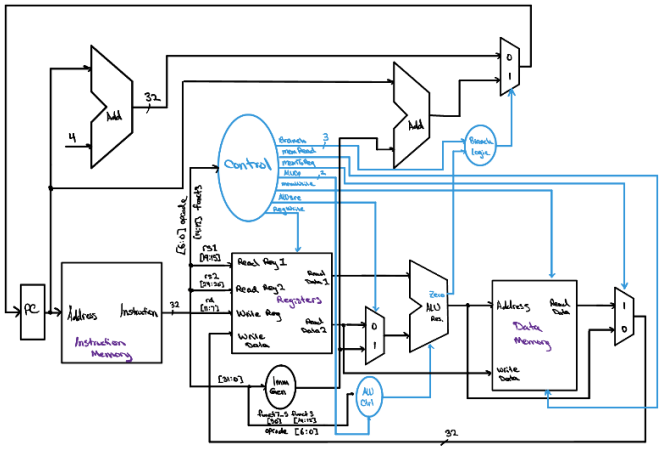

Figure 1: System Architecture

2.2 Module Hierarchy

```
sc_cpu_top_level
├── sc_cpu_control          Main control unit (combinational, opcode-decoded)
├── alu_control             ALU control unit (funct3/funct7 decode)
└── sc_cpu_datapath
    ├── instruct_mem        Read-only instruction memory (loaded from program.mem)
    ├── imm_gen             Immediate generator (I, S, SB, U, UJ formats)
    ├── reg_file            32x32-bit register file (2 read ports, 1 write port)
    ├── mux_2x1             ALU source mux; write-back mux; branch/increment mux (×3)
    ├── alu_full            32-bit ALU (ripple-carry, parameterized)
    │   ├── alu_slice       1-bit ALU slice (bits [30:0])
    │   │   ├── mux_2x1     Output select (ALU result vs. pass-through)
    │   │   ├── mux_4x1     Operation select (add/sub, and, or, slt)
    │   │   └── full_adder  1-bit full adder
    │   └── alu_msb         MSB ALU slice (overflow detection, setLess)
    │       ├── mux_2x1     Output select (ALU result vs. pass-through)
    │       ├── mux_4x1     Operation select (add/sub, and, or, slt)
    │       └── full_adder  1-bit full adder (MSB, feeds overflow logic)
    ├── data_mem            Clocked data memory (word-aligned, read/write enable)
    └── ripple_carry_adder  PC + 4 adder; PC + immediate adder (×2)
```

3. Verification Methodology and Testbench

The testbench (`tb_sc_cpu_top_level`) is a self-checking, top-level integration testbench targeting Icarus Verilog (`iverilog -g2012`). Verification uses two independent oracles so that a bug in either the design or the checking code is unlikely to escape undetected.

3.1 Key Parameters

Parameter	Value / Purpose
CLK_PERIOD	10 ns — drives the always block that toggles clk
NUM_INSTR	30 — halt boundary; simulation stops when ref_pc word index >= 30
DMEM_WORDS	256 — number of 32-bit data-memory words checked each step
IMEM_WORDS	256 — reference instruction-memory size (mirrored from program.mem)
MAX_STEPS	1000 — runaway watchdog limit

3.2 Lockstep Golden Model

A behavioral RV32I-subset simulator (task `ref_step`) decodes and executes one RV32I instruction per call, maintaining its own `ref_regs[0:31]`, `ref_dmem[0:255]`, and `ref_pc`. It runs one instruction ahead of each DUT clock edge, encoding the correct architectural semantics. After every `posedge clk` the task `check_state` compares the DUT PC (`dut.datapath.pc`), all 32 register-file entries (`dut.datapath.registers.RF[k]`), and all 256 data-memory words (`dut.datapath.data_memory.memory[k]`) against the reference. Any mismatch is logged immediately with step number and field name.

3.3 Final-State Oracle

The tasks `load_expected` and `check_final` encode a hand-derived, independently computed table of the expected end-of-program register and memory state, checked once after halt. Because it is produced by a completely different method from the lockstep model (manual trace), agreement among DUT, lockstep model, and this table gives high confidence that a latent bug in the lockstep model itself has not masked a DUT defect.

3.4 Supported Instruction Subset

The golden model (task `ref_step`) decodes and executes the following RV32I instructions:

Opcode group	Instructions	Notes
OPC_RTYPE (0110011)	add, sub, and, or, slt	funct7[5] selects sub; signed compare for slt
OPC_ITYPE (0010011)	addi, andi, ori, slti	12-bit sign-extended immediate
OPC_LOAD (0000011)	lw	Byte address = rs1 + imm_i; word-aligned
OPC_STORE (0100011)	sw	Byte address = rs1 + imm_s; word-aligned
OPC_BRANCH (1100011)	beq, bne, blt	PC-relative; signed compare for blt

3.5 Halt Condition and Watchdog

The main execution loop terminates when `ref_pc[31:2]` >= `NUM_INSTR` (30). Because the DUT uses the same program image and identical control flow, it reaches the same halt condition on the same clock edge. A global watchdog (`initial` block with absolute delay) triggers `$finish` after `CLK_PERIOD * (MAX_STEPS + 50)` nanoseconds if the loop has not already exited, preventing the simulator from hanging on a runaway design.

3.6 Waveform Dump

The testbench calls `$dumpfile("waveforms/dump.vcd")` and `$dumpvars(0, tb_sc_cpu_top_level)` in an `initial` block, producing a Value Change Dump covering the full hierarchy. The VCD is viewable in EPWave (EDA Playground) or any compatible waveform viewer such as GTKWave.

3.7 Per-Instruction Trace and Disassembler

After each step the task `trace_print` emits a formatted line containing the step counter, PC (hex), instruction word (hex), a pseudocode disassembly produced by the task `disasm`, and a plain-English description of the architectural effect (register write, store, branch taken/not taken, or no state change). The `disasm` task covers all opcodes in the supported subset and falls back to `.word 0xXXXXXXXX` for any unrecognized encoding.

3.8 Hierarchical References

The testbench accesses DUT internal state directly via hierarchical dot-notation paths, which must match the actual module and signal names in the RTL:

```
dut.datapath.pc                          — program counter
dut.datapath.registers.RF[k]             — register file array (k = 0..31)
dut.datapath.data_memory.memory[k]       — data memory array (k = 0..255)
```

If the RTL uses different instance or signal names, these three paths must be updated accordingly for the testbench to compile and link.

3.9 Verification Task Reference

Task	Role
disasm	Decode one 32-bit instruction word into readable pseudocode.
ref_step	Decode and execute one instruction in the golden model, updating ref_regs, ref_dmem and ref_pc.
check_state	Compare DUT PC, all 32 registers, and all 256 data-memory words against the reference.
trace_print	Emit a human-readable per-instruction trace line.
dump_final_state	Print the complete register file and non-zero data memory at halt.
load_expected	Populate the hand-derived final-state table.
check_final	Compare DUT final state against the hand-derived table.
report_execution_stats	Print retired-instruction count, cycle count, and cycles per retire in the same format the pipelined testbench uses, so the two can be read side by side.

4. Program Under Test

The 30-instruction program (program.mem) is loaded into both the DUT instruction memory and the reference model at simulation start via $readmemh. It is generated by programs/build_program.py, which holds the instruction list in source form and writes the identical image into both this design and the pipelined design, so the two cannot drift apart. Do not hand-edit program.mem. The execution trace below shows every committed instruction, the disassembled pseudocode, and the architectural effect. For detailed waveshape, see Appendix A.

4.1 Execution Trace

Step	PC	Encoding	Pseudocode	Effect
1	0x00000000	0x00500093	addi x1, x0, 5	x1 ← 0x00000005 (5)
2	0x00000004	0xFFD00113	addi x2, x0, -3	x2 ← 0xFFFFFFFD (-3) — negative immediate
3	0x00000008	0x001101B3	add x3, x2, x1	x3 ← 0x00000002 (2)
4	0x0000000C	0x40308233	sub x4, x1, x3	x4 ← 0x00000003 (3)
5	0x00000010	0x001172B3	and x5, x2, x1	x5 ← 0x00000005 (5)

Step	PC	Encoding	Pseudocode	Effect
6	0x00000014	0x00416333	or x6, x2, x4	x6 ← 0xFFFFFFFF (-1)
7	0x00000018	0x001123B3	slt x7, x2, x1	x7 ← 0x00000001 (1) — -3 < 5 true, signed
8	0x0000001C	0xFFF0A413	slti x8, x1, -1	x8 ← 0x00000000 (0) — 5 < -1 false, signed
9	0x00000020	0x0030F493	andi x9, x1, 3	x9 ← 0x00000001 (1)
10	0x00000024	0x0080E513	ori x10, x1, 8	x10 ← 0x0000000D (13)
11	0x00000028	0x04000593	addi x11, x0, 64	x11 ← 0x00000040 (64) — base address
12	0x0000002C	0x00A5A023	sw x10, 0(x11)	dmem[16] ← 0x0000000D
13	0x00000030	0x0005A603	lw x12, 0(x11)	x12 ← 0x0000000D (13)
14	0x00000034	0x001606B3	add x13, x12, x1	x13 ← 0x00000012 (18)
15	0x00000038	0x00D5A223	sw x13, 4(x11)	dmem[17] ← 0x00000012
16	0x0000003C	0x0045A703	lw x14, 4(x11)	x14 ← 0x00000012 (18)
17	0x00000040	0x00D70463	beq x14, x13, +8	beq: x14=18 == x13=18 ← TAKEN ← 0x00000048
—	0x00000044	0x06300093	addi x1, x0, 99	dead code — skipped by the branch at step 17
18	0x00000048	0x00400793	addi x15, x0, 4	x15 ← 0x00000004 (4)
19	0x0000004C	0x00900813	addi x16, x0, 9	x16 ← 0x00000009 (9)
20	0x00000050	0x0107C463	blt x15, x16, +8	blt: 4 < 9 ← TAKEN ← 0x00000058
—	0x00000054	0x06300113	addi x2, x0, 99	dead code — skipped by the branch at step 20
21	0x00000058	0x00F84463	blt x16, x15, +8	blt: 9 < 4 false — NOT taken
22	0x0000005C	0x00F80463	beq x16, x15, +8	beq: 9 == 4 false — NOT taken
23	0x00000060	0x00300893	addi x17, x0, 3	x17 ← 0x00000003 (3) — loop counter
24	0x00000064	0x01190933	add x18, x18, x17	x18 ← 0x00000003 (3) — loop pass 1
25	0x00000068	0xFFF88893	addi x17, x17, -1	x17 ← 0x00000002 (2)
26	0x0000006C	0xFE089CE3	bne x17, x0, -8	bne: 2 != 0 ← TAKEN ← 0x00000064 (backward)
27	0x00000064	0x01190933	add x18, x18, x17	x18 ← 0x00000005 (5) — loop pass 2
28	0x00000068	0xFFF88893	addi x17, x17, -1	x17 ← 0x00000001 (1)
29	0x0000006C	0xFE089CE3	bne x17, x0, -8	bne: 1 != 0 ← TAKEN ← 0x00000064
30	0x00000064	0x01190933	add x18, x18, x17	x18 ← 0x00000006 (6) — loop pass 3
31	0x00000068	0xFFF88893	addi x17, x17, -1	x17 ← 0x00000000 (0)

Step	PC	Encoding	Pseudocode	Effect
32	0x0000006C	0xFE089CE3	bne x17, x0, -8	bne: 0 != 0 false — NOT taken, loop exits
33	0x00000070	0x07B00013	addi x0, x0, 123	x0 write discarded (I-type)
34	0x00000074	0x00208033	add x0, x1, x2	x0 write discarded (R-type)

Instructions at PCs 0x44 and 0x54 are dead code: the taken branches at steps 17 and 20 cause execution to skip them entirely. Both write the value 99 to a register that is checked in the final-state oracle (x1 and x2), so if branch behaviour ever regressed and these executed, the end-of-program table in Section 5.3 would fail. Steps 24 through 32 are the three iterations of a backward-branch loop occupying only three instruction words, which is why 34 instructions retire from a 30-word image.

5. Verification Results

5.1 Lockstep Check

The lockstep check fires after every clock edge and compares the DUT PC, all 32 registers, and all 256 data-memory words against the golden model. Zero mismatches were recorded across all 34 retired instructions.

5.2 Final-State Oracle

The hand-derived oracle was compared against the DUT final state after halt. Zero mismatches were recorded. The complete expected and observed state is shown below.

5.3 Final Register File

Register	Expected (hex)	Observed (hex)	Comment
x0	0x00000000	0x00000000	Hardwired zero — writes at steps 33 and 34 discarded
x1	0x00000005	0x00000005	addi x1, x0, 5 — NOT 99, proving the step 17 branch flushed
x2	0xFFFFFFFD	0xFFFFFFFD	addi x2, x0, -3 — NOT 99, proving the step 20 branch flushed
x3	0x00000002	0x00000002	add x3, x2, x1 (-3 + 5)
x4	0x00000003	0x00000003	sub x4, x1, x3 (5 - 2)
x5	0x00000005	0x00000005	and x5, x2, x1 (-3 & 5)
x6	0xFFFFFFFF	0xFFFFFFFF	or x6, x2, x4 (-3 | 3)
x7	0x00000001	0x00000001	slt x7, x2, x1 — signed, true
x8	0x00000000	0x00000000	slti x8, x1, -1 — signed, false
x9	0x00000001	0x00000001	andi x9, x1, 3
x10	0x0000000D	0x0000000D	ori x10, x1, 8

Register	Expected (hex)	Observed (hex)	Comment
x11	0x00000040	0x00000040	addi x11, x0, 64 — data memory base
x12	0x0000000D	0x0000000D	lw x12, 0(x11) — reads back the store at step 12
x13	0x00000012	0x00000012	add x13, x12, x1
x14	0x00000012	0x00000012	lw x14, 4(x11) — reads back the store at step 15
x15	0x00000004	0x00000004	addi x15, x0, 4
x16	0x00000009	0x00000009	addi x16, x0, 9
x17	0x00000000	0x00000000	Loop counter, decremented to zero
x18	0x00000006	0x00000006	Loop sum: 3 + 2 + 1
x19–x31	0x00000000	0x00000000	Not written

5.4 Final Data Memory (non-zero words)

Word index	Byte address	Expected (hex)	Observed (hex)	Source instruction
16	0x40	0x0000000D	0x0000000D	sw x10, 0(x11) at step 12
17	0x44	0x00000012	0x00000012	sw x13, 4(x11) at step 15

6. Functional Coverage

6.1 Coverage Matrix

Coverage Point	Status	Notes
R-type: add	COVERED	Steps 3, 14, 24, 27, 30, 34
R-type: sub	COVERED	Step 4
R-type: and	COVERED	Step 5 — negative operand
R-type: or	COVERED	Step 6 — negative operand, result -1
R-type: slt (true result)	COVERED	Step 7 — -3 < 5 signed
R-type: slt (false result)	COVERED	Implied by step 8 slti; slt false not directly hit
I-type: addi (positive imm)	COVERED	Steps 1, 11, 18, 19, 23, 33
I-type: addi (negative imm)	COVERED	Steps 2 (-3), 25, 28, 31 (-1)
I-type: andi	COVERED	Step 9
I-type: ori	COVERED	Step 10
I-type: slti	COVERED	Step 8 — previously a coverage gap, now closed

Coverage Point	Status	Notes
Signed comparison, negative operands	COVERED	Steps 7, 8, and blt at 20/21
Load: lw	COVERED	Steps 13, 16 — both read back a prior store
Store: sw	COVERED	Steps 12, 15
Load-after-store to same address	COVERED	Steps 12/13 and 15/16
Branch: beq taken	COVERED	Step 17
Branch: beq not taken	COVERED	Step 22
Branch: bne taken	COVERED	Steps 26, 29 — both backward
Branch: bne not taken	COVERED	Step 32 — loop exit
Branch: blt taken	COVERED	Step 20
Branch: blt not taken	COVERED	Step 21
Backward branch / loop	COVERED	Steps 24–32, three iterations
x0 write suppression, I-type	COVERED	Step 33: addi x0, x0, 123
x0 write suppression, R-type	COVERED	Step 34: add x0, x1, x2
Registers x19–x31 written	NOT COVERED	Only x0–x18 are touched
Data memory outside words 16–17	NOT COVERED	Only two words are written

6.2 Dead-Code Lines

Two instructions are never executed: addi x1, x0, 99 at PC 0x44 and addi x2, x0, 99 at PC 0x54, skipped by the taken branches at steps 17 and 20. Both are present in the program image and decoded by the reference disassembler but contribute no architectural effect. Their targets were chosen deliberately: x1 and x2 are read by the final-state oracle, so the dead code doubles as a check that the branches really did skip them.

7. Issues and Recommendations

7.1 Open Issues

#	Issue	Details
1	Upper registers unexercised	Registers x19 through x31 are never written. The register file is a uniform array, so a fault affecting only the upper half is unlikely, but it is untested.
2	Data memory sparsely exercised	Only word indices 16 and 17 are written. Index 0 and the upper boundary at index 255 are never touched, so the address decoder is only lightly stressed.
3	Program is pipeline-oriented	Roughly half the program targets hazards this design does not have. That makes it a correct but not especially efficient test for a single-cycle CPU, and the coverage it does deliver is a by-product rather than the design intent. The task report_execution_stats prints its cycle counts in the same format the pipelined testbench uses, so the two designs can be compared side by side.

7.2 Recommendations

- Supplement program.mem with a short follow-on sequence that covers slti and at least one upper register (x17+).
- Consider adding a `` `timescale `` directive to each RTL module to eliminate the Icarus Verilog timing-precision warning that appears when mixed-timescale designs are compiled together.
- Add a fault-injection run: force one register to a wrong value mid-simulation and confirm that the lockstep check flags it immediately.
- Extend data-memory testing to include word index 0 and the upper boundary (index 255) to stress the address decoder.

8. Conclusion

The single-cycle RISC-V CPU (sc_cpu_top_level) passes all verification checks on the 30-instruction test program, retiring 34 instructions with zero errors. The dual-oracle testbench reported zero mismatches from the lockstep comparison and zero from the independent hand-derived final-state check. The design correctly implements every instruction in the supported subset, including slti, which the previous program did not reach. Signed comparison with negative operands, negative immediates, load-after-store to the same address, a backward-branch loop, all six branch cases, and x0 write suppression from both instruction formats were all exercised. The remaining coverage gaps are minor and are addressed by the recommendations in Section 7.

Appendix A. Simulation Waveform

The waveform below shows the DUT signal activity captured during the simulation run: 34 instructions retired in 34 clock cycles, one per cycle, which is the defining property of a single-cycle design. Signals are grouped by function: clock, reset, and pc (yellow), instruction decode (violet), control (green), data words (red), and ALU control signals (orange).